

Functions & List Power Tools

Post-Assessment 1 · 10 questions · 33 points

Name: _____

Date: _____

Score: / 33

This is a **post-assessment**. It shows what you learned in this unit. Try every question, and check your answers if you have time.

Work on your own, without a computer. You have about **40 minutes**. When a question asks what a program prints, write exactly what Python would print, one line of output per answer line. If the code would crash, write the name of the error (for example, `TypeError`). Show your thinking; partial credit is given.

1. Two answers at once

2 points

```
def split_name(full):  
    parts = full.split()  
    return parts[0], parts[-1]  
  
first, last = split_name("Grace Brewster Hopper")  
print(last, first)  
print(split_name("Alan Turing"))
```

Write exactly what the program prints.

2. Greeting Toolkit

3 points

```
def greet(name="friend", punctuation="!"):
    return "Hello, " + name + punctuation

def greet_twice(name):
    return greet(name) + " " + greet(name, punctuation="?")

print(greet())
print(greet("Sam", "."))
print(greet_twice("Lee"))
```

Write exactly what the program prints.

3. Whose variable?

2 points

```
level = 1

def next_level(n):
    level = n + 1
    return level

print(next_level(7))
print(level)
```

Write exactly what the program prints.

4. Test first

4 points

A student wrote these tests **before** writing the function. A “long word” has 7 or more letters.

TESTS

```
assert is_long_word("elephant") == True
assert is_long_word("cat") == False
```

- (a) If `is_long_word("cat")` wrongly returned `True`, what would happen when the tests run? (1 pt)
What happens to a test that passes?

- (b) Write `is_long_word(word)` with a docstring so that both tests pass. (2 pts)

- (c) Write one more good `assert` test for `is_long_word`. (1 pt)

5. Bug-Proof Calculator

3 points

```
def total_length(words):  
    """Return the total number of letters in all the words."""  
    total = 0  
    for w in words:  
        total = len(w)  
    return total  
  
assert total_length(["hi", "cat"]) == 5
```

(a) What does `total_length(["hi", "cat"])` actually return? (1 pt)

(b) What happens when the `assert` line runs? (1 pt)

(c) Describe the fix. (1 pt)

6. Seating chart

3 points

```
seats = [ ["Ana", "Ben"],  
          ["Cy", "Dee"],  
          ["Eli", "Fay"]]  
print(seats[2])  
print(seats[0][1])  
print(seats[1][0] + seats[2][-1])
```

Write exactly what the program prints.

7. List power tools

4 points

```
cities = ["Oslo", "Lima", "Cairo"]
temps = [3, 19, 30]
for city, t in zip(cities, temps):
    print(city, t)

animals = ["zebra", "ox", "lion"]
print(sorted(animals))
print(sorted(animals, key=len))
for i, a in enumerate(animals):
    print(i, a)
```

Write exactly what the program prints.

8. Unique dice rolls

3 points

```
rolls = [3, 6, 3, 1, 6, 6, 2]
seen = set(rolls)
print(len(rolls), len(seen))
seen.add(6)
seen.add(5)
print(len(seen))
print(4 in seen)
```

Write exactly what the program prints.

9. One-line list builders

4 points

(a) What is `[n + 10 for n in [5, 6, 7]]`?

(1 pt)

(b) What is `[w for w in ["apple", "kiwi", "plum", "mango"] if len(w) == 4]`?

(1 pt)

(c) Rewrite this loop as **one** list comprehension:

(2 pts)

```
shouted = []
for word in words:
    if len(word) > 3:
        shouted.append(word.upper())
```

10. Cheapest First

5 points

Each item is stored as `[name, price]`:

```
items = [["juice", 3], ["chips", 2], ["cookie", 1]]
```

Write a function `cheapest_first(items)` that **returns** a list of just the item names, from lowest price to highest. For the list above it returns `["cookie", "chips", "juice"]`. Then write one `assert` test for it.

END OF ASSESSMENT